arXiv:2411.16934v3 [cs.CV] 19 Dec 2025

Online Episodic Memory Visual Query Localization with Egocentric Streaming Object Memory

Zaira Manigrasso[1*] Matteo Dunnhofer[1,3*] Antonino Furnari[2*]
Moritz Nottebaum[1] Antonio Finocchiaro[2] Davide Marana[1] Rosario Forte[2]
Giovanni Maria Farinella[2⋆] Christian Micheloni[1⋆]

[1] University of Udine, Italy [2] University of Catania, Italy [3] York University, Canada

Abstract

Episodic memory retrieval enables wearable cameras to recall objects or events previously observed in video. However, existing formulations assume an "offline" setting with full video access at query time, limiting their applicability in real-world scenarios with power and storage-constrained wearable devices. Towards more application-ready episodic memory systems, we introduce Online Visual Query 2D (OVQ2D), a task where models process video streams online, observing each frame only once, and retrieve object localizations using a compact memory instead of full video history. We address OVQ2D with ESOM (Egocentric Streaming Object Memory), a novel framework integrating an object discovery module, an object tracking module, and a memory module that find, track, and store spatio-temporal object information for efficient querying. Experiments on Ego4D demonstrate ESOM's superiority over other online approaches, though OVQ2D remains challenging, with top performance at only 4% success. ESOM's accuracy increases markedly with perfect object tracking (31.91%), discovery (40.55%), or both (81.92%), underscoring the need of applied research on these components.

1. Introduction

Day-to-day experiences often witness lapses in human memory. We may misplace objects within our homes (like searching for a passport), lose track of completed tasks (wondering if salt was added), forget the locations of past activities (where were tickets purchased?), and overlook the status of objects in our surroundings (did I leave the garage door open?). Wearable cameras observing human activities from an egocentric perspective have the potential to assist the recall of such episodic memories [47] in a wide range of scenarios by continuously keeping track of observed objects and events to alleviate cognitive overload and support the execution of daily tasks [36].

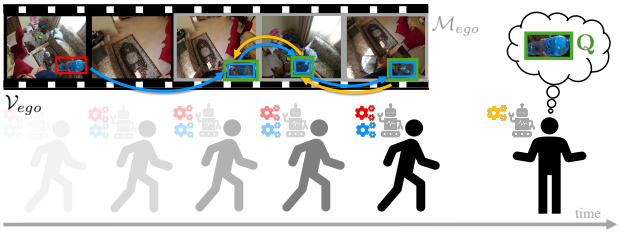

Figure 1. **Online visual query localization via object memorization and retrieval.** We tackle the problem of online episodic memory and propose ESOM ( ), an architecture that processes an egocentric video ($\mathcal{V}_{ego}$) online and only once to detect ( ), track ( ), and memorize ($\mathcal{M}_{ego}$) user-relevant objects and frames. When retrieving a visual query (Q), ESOM ( ) searches ( ) its memory ($\mathcal{M}_{ego}$) for the most recent instance where the query was spatio-temporally localized. By avoiding video storage, ESOM optimizes both memory usage and retrieval speed.

Towards this direction, the Visual Queries 2D (VQ2D) task has been introduced with the Episodic Memory (EM) benchmark of the Ego4D dataset [22]. VQ2D is formulated as follows: given an image patch of an object query and a long egocentric video, identify a *response track*, the precise spatio-temporal location of the last appearance of the object in the video. VQ2D has fostered the development of algorithms for episodic memory retrieval in recent years [7, 17, 26, 29, 33, 35, 38, 39, 50, 51]. We observe that the current formulation of VQ2D provides for solutions to operate in an *offline* manner. Specifically, given a visual query at frame t, methods search within $\mathcal{V}_{ego}$ scanning it from current frame t backward up to the moment in which the object is found. Arguably, this approach is limited in

* denotes equal contribution. ⋆ denotes equal supervision.

practical application scenarios in which an always-on wearable device processes several hours of video per day. In fact, offline VQ2D requires the burdensome storage of all past video and entails significant computational overhead when retrieving multiple queries, requiring the scan of the same video several times. We advocate that EM systems should be able to work in an *online* fashion, with limited budgets for computation and storage, reflecting the need to ultimately deploy algorithms on wearable devices.

We hence propose a new task formulation for EM, Online Visual Query 2D (OVQ2D), in which models are required to work online, processing video frames only once, storing relevant information in a compact memory, and relying on such memory to answer queries whenever they are made by the user. Similarly to offline VQ2D, methods need to return a series of bounding boxes and corresponding past frames where the queried object appears. This enables clear spatio-temporal localization within its visual context, forming an interpretable episode of its last known location. To assess practical applicability, we compare memory size and computation time, besides mere retrieval performance. Note that this online formulation brings several challenges, beyond the need for efficient systems. Specifically: 1) online systems have to identify objects which could be potentially queried in the future without any prior knowledge of their identity, in contrast to offline systems which begin the object search only *after* the query is known; 2) online systems should be able to keep track of the different occurrences of a given object instance and store appropriate response tracks and related frames *in advance*, in contrast to offline systems which initiate a separate search for each query; 3) Online systems should be able to associate each of the potential response tracks with appropriate representations to enable future retrieval, in contrast to offline systems, which can directly compare the query patch with object candidates during the search process.

To tackle OVQ2D, we propose ESOM (Egocentric Stream Object Memory), a principled architecture built on state-of-the-art object detection and tracking, leveraging decades of progress in computer vision. This formulation adopts an application-driven focus, prioritizing practical deployment and benchmarking over algorithmic novelty. ESOM processes egocentric video frames online, maintaining a compact, interpretable, and dynamic memory of past objects (see Figure 1). This is achieved with three main components: 1) an *object discovery* module detecting objects of interest in each input frame; 2) an online, long-term *visual object tracker* tracking the discovered objects in the video; 3) a dynamic *object memory* which stores past observations in the form of object tracks with associated visual representations. The object memory is built through an *object memory population* process that coordinates the object discovery and visual tracking modules. It can then be queried by a *query retrieval and localization* process, which efficiently searches the memory and returns the corresponding response track in response to a user query.

Experiments on Ego4D [22] demonstrate the effectiveness of ESOM over other online methods. Results also show that OVQ2D is challenging, with the best method achieving a success rate of only $\sim$4%. This modest performance is mainly due to the limitations of current object detection and tracking on real-world egocentric videos. When using oracle detection or tracking, ESOM's success rate rises to 31.91% and 40.55%, respectively, and reaches 81.92% with both. These results also shed light on the limited performance of current object detection and tracking algorithms when employed in real-world applications, suggesting that more efforts are needed to improve their practical applicability. Unlike standard evaluations that assess these components in isolation, our study offers a competitive benchmark that can be used to assess progress and applicability in real-world downstream tasks.

Overall, the contributions of this work are as follows: 1) we introduce OVQ2D, an online formulation for episodic memory retrieval; 2) we propose ESOM, a novel architecture based on continuous object discovery, tracking, memory update, and query, which is shown to outperform offline methods under competitive memory budgets and brings benefits in terms of runtime; 3) we offer a benchmark based on Ego4D [22] for OVQ2D, which we also show as a valuable resource to evaluate real-world performance of object detection and tracking algorithms.

2. Related Work

Episodic Memory Visual Query Localization. The task of Visual Query Localization was initially formalized in [22], along with a baseline cascade solution that first searches for the query patch in every frame by means of a Siamese R-CNN [48], and then tracks it forward and backward in time with a short-term tracker [5] to obtain the temporal extent of the response track. Based on this formulation, a variety of methodologies explored different key factors such as balancing positive and negative examples [50], addressing domain and task biases [51], improving feature extractors [7, 35], and developing end-to-end single-stage frameworks [26] to improve performance and inference speed. Despite the progress, all the aforementioned works are based on the offline assumption. In this paper, we formulate a novel Online Visual Query 2D (OVQ2D) task and offer a first approach and benchmark to tackle the problem.

Object Memory and Cognition. Object memory representations have been investigated in robotics [25] to support object permanence in manipulation tasks, and in egocentric

For instance, a 24-hour 8-bit RGB video with 1408x1408 resolution captured at 20FPS [44] can take up to 3.43 TB of uncompressed memory.

vision [37] to mimic human spatial cognition in keeping track of object instances when they are out-of-sight. Recent work [15, 21], in particular, explored the use of an object memory to address video understanding tasks such as generic question answering [15] and comprehension of object-based activities in long egocentric videos [21]. Unlike the aforementioned works, our method is the first to propose and tackle online episodic memory with an object memory, which raises significant challenges due to the need to account for *all* objects appearing in the scene and accurately track object instances in the video.

Online Processing in Egocentric Vision. Previous work tackled the design of algorithms and systems to support online processing, a fundamental requirement for real-world applications on wearable devices [2, 9, 10, 18–20, 24, 41, 42, 45, 55], in contrast to a whole line of egocentric vision tasks such as action recognition [4, 6, 16], and temporal action detection [31, 53] which assume that the whole video is available at inference time. Episodic memory retrieval tasks, as initially formulated in [22] are based on the "offline" assumption. Limited amount of work considered alternative formulations resembling an online setting [3], and a systematic investigation on the topic is still missing. We address the problem of online episodic memory retrieval targeting the VQL task. Our system achieves online processing abilities by using visual object tracking and a compact queryable memory module to store past experiences.

3. Online Visual Query 2D Localization

We define Online Visual Query 2D (OVQ2D) as follows. Given a streaming egocentric video $\mathcal{V}_{ego} = \{\mathbf{F}_t\}$ of RGB frames $\mathbf{F}_t$ indexed by time t, and a query image patch $\mathbf{Q}$ representing the visual appearance of the query object, the goal is to retrieve a visual response track:

$$\mathbf{r} = \{(\mathbf{F}_s, \mathbf{b}_s), (\mathbf{F}_{s+1}, \mathbf{b}_{s+1}), \dots, (\mathbf{F}_e, \mathbf{b}_e)\}, \quad (1)$$

which is an ordered sequence of (frame, bounding box) pairs $(\mathbf{F}_j, \mathbf{b}_j)$, where $s \leq j \leq e \leq t$. This track determines the spatio-temporal position of the object's last appearance in $\mathcal{V}_{ego}$, with s and e denoting starting and ending frames, and each box $\mathbf{b}_j$ corresponding to frame $\mathbf{F}_j$. This formulation allows for the visualization of $\mathbf{Q}$'s spatio-temporal localization within its surrounding context, generating a detailed video snippet that helps users enhance their natural episodic memory. As in [22], the goal is to find $\mathbf{r}$ given $\mathbf{Q}$, however, critically, our formulation requires methods to process $\mathcal{V}_{ego}$ in a streaming fashion, observing frames $\mathbf{F}_t$ only once. To support future queries, methods can store information in a compact memory. Figure 1 provides an illustration of our problem formulation.

4. Egocentric Streaming Object Memory

To tackle OVQ2D, we propose the Egocentric Streaming Object Memory (ESOM) architecture. ESOM is based on the object memory $\mathcal{M}_{ego}$, a dynamic data structure that memorizes relevant object instances appearing in $\mathcal{V}_{ego}$. The data structure is populated and queried with two processes: (i) the *object memory population* algorithm that builds $\mathcal{M}_{ego}$ online and continuously as $\mathcal{V}_{ego}$ is being captured; (ii) the *query retrieval and localization* algorithm that retrieves the visual response track $\mathbf{r}$ related to the query $\mathbf{Q}$ from $\mathcal{M}_{ego}$, whenever needed by the camera wearer. The continuous interaction between these components allows to dynamically create, update, and query the memory of previously observed objects to support online episodic memory retrieval without the need of storing all the frames of $\mathcal{V}_{ego}$.

4.1. Object Memory

Representation. We define the object memory as a set

$$\mathcal{M}_{ego} = \{\mathbf{O}_1, \dots, \mathbf{O}_i, \dots, \mathbf{O}_N\} \quad (2)$$

of representations $\mathbf{O}_i$ of the spatio-temporal locations of objects in the frames $\mathbf{F}_t$ of $\mathcal{V}_{ego}$. The indexes $1 \leq i \leq N$ denote object instance unique IDs. While the representation of $\mathbf{O}_i$ can be arbitrarily abstract, in practice, we set $\mathbf{O}_i$ to be a list $\mathbf{O}_i = \{\mathbf{o}_{i,t}\}$ of records temporally indexed by t, with each record having the following form

$$\mathbf{o}_{i,t} = [t, \mathbf{b}_{i,t}, \mathbf{F}_t, c_{i,t}] \quad (3)$$

where $\mathbf{b}_{i,t} = [b^x_{i,t}, b^y_{i,t}, b^w_{i,t}, b^h_{i,t}]$ denotes the object bounding box, $\mathbf{F}_t$ is the associated RGB frame at time t, and $c_{i,t}$ is a binary label indicating whether the visual content within $\mathbf{b}_{i,t}$ is relevant (i.e., the appearance is distinctive) for the retrieval process. If $\mathbf{o}_{i,t}$ is not visible in $\mathbf{F}_t$, then $\mathbf{o}_{i,t} = \emptyset$. This formulation of the object memory provides an indexed, lightweight, compact, human-readable, and easily queryable representation of objects observed in $\mathcal{V}_{ego}$ up to the current frame $\mathbf{F}_t$. Figure 2 visualizes the representation of $\mathcal{M}_{ego}$ with respect to the information of $\mathcal{V}_{ego}$.

Interface. Interaction with $\mathcal{M}_{ego}$ is achieved through two operations: `.read()`, and `.write()`. The `.read()` operation is used to access information on specific objects stored in $\mathcal{M}_{ego}$. Following the object-oriented programming notation, we design `.read()` as an overloaded method that can take as input either an object ID i or a frame ID t. $\mathcal{M}_{ego}$`.read(`i`)` returns the list of spatio-temporal localizations related to object instance $\mathbf{O}_i$, whereas the $\mathcal{M}_{ego}$`.read(`t`)` operation returns the spatial information of all objects $\mathbf{O}_i$ that are memorized at time t:

$$\{\mathbf{o}_{j,t}\} = \mathcal{M}_{ego}\texttt{.read}(t), 1 \leq j \leq N \quad (4)$$

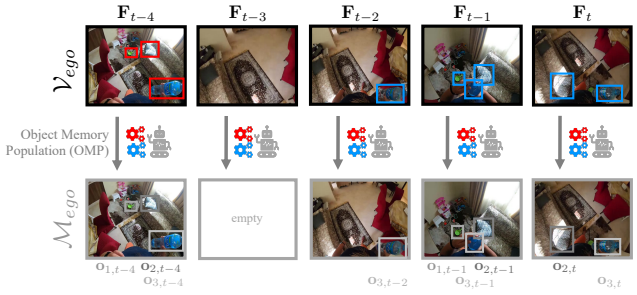

Figure 2. **From an egocentric video to a memory of objects.** ESOM injects the visual information in each frame $\mathbf{F}_t$ of a video $\mathcal{V}_{ego}$ into $\mathcal{M}_{ego}$, an object memory represented as a dynamic list $\mathbf{O}_i$ of tuples $\mathbf{o}_{i,t}$ composed of instance-based, frame-level bounding boxes $\mathbf{b}_{i,t}$, related frames $\mathbf{F}_t$, and relevance labels $c_{i,t}$. $\mathcal{M}_{ego}$ is built by an Object Memory Population (OMP) algorithm which processes ( ) frames $\mathbf{F}_t$ online.

The `.write()` operation allows to store new object instances or update existing ones. As before, this operation can be called with different input parameters. The syntax

$$\mathcal{M}_{ego}.\texttt{write}\left(\left[t, \mathbf{b}_{k,t}, \mathbf{F}_t, c_{k,t}\right]\right) \tag{5}$$

stores with ID k (not yet present in $\mathcal{M}_{ego}$) a new object instance $\mathbf{O}_k$ by inserting timestamp t, the bounding box $\mathbf{b}_{k,t}$, the associated frame $\mathbf{F}_t$, if it is not already in $\mathcal{M}_{ego}$, and the label $c_{k,t}$. The `write` operation can also be called as

$$\mathcal{M}_{ego}.\texttt{write}\left(\left[t, \mathbf{b}_{i,t}, \mathbf{F}_t, c_{i,t}\right]\right) \tag{6}$$

to add t, $\mathbf{b}_{i,t}$, $\mathbf{F}_t$, and $c_{i,t}$ to object instance $\mathbf{O}_i$ at time t (i.e., we update the representation of $\mathbf{O}_i$). If $\mathbf{F}_t$ is already stored in $\mathcal{M}_{ego}$, no duplicate is stored. When `.write()` updates an object, it checks for a break from the previous timestamp $\hat{t}$ (i.e., $t - \hat{t} > 1$). If occurs, frames $\mathbf{F}_{\bar{t}}, \bar{t} \leq \hat{t}$ linked to the prior sequence of contiguous boxes are deleted, unless they belong to another memorized object.

4.2. Object Memory Population (OMP)

The object memory $\mathcal{M}_{ego}$ is created and updated by a dedicated Object Memory Population (OMP) algorithm designed to process $\mathcal{V}_{ego}$ online. At each step t, this algorithm receives as input the latest available frame $\mathbf{F}_t$ and updates the object memory by executing an Object Tracking (OT) module and an Object Discovery (OD) module. A schematic visualization of the two modules within the OMP algorithm is given in Figure 3.

Object Tracking Module. The OT module is responsible for updating the localization of objects stored in $\mathcal{M}_{ego}$ at previous time steps, maintaining their unique IDs. At each frame $\mathbf{F}_t$, the OT module first retrieves all previously memorized objects $\mathbf{O}_i$ via $\mathcal{M}_{ego}.\texttt{read}(i)$ $\forall i$, then estimates $\mathbf{b}_{i,t}$, the position of each object i in frame t by

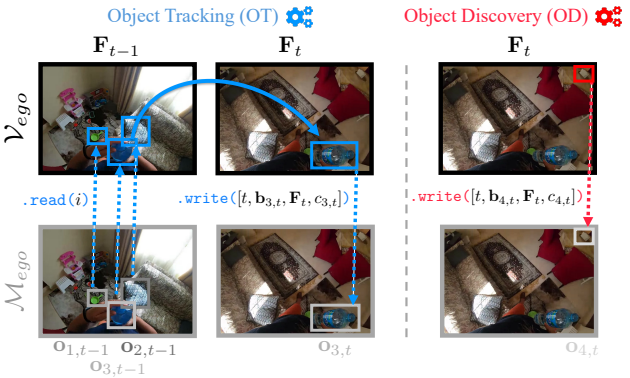

Figure 3. **Memory population by object tracking and discovery.** The Object Tracking (OT) ( ) reads objects $\mathbf{o}_{i,t-1}$ related to the last frame $\mathbf{F}_{t-1}$ from $\mathcal{M}_{ego}$ and updates their position in current frame $\mathbf{F}_t$. In parallel, new objects are detected by the Object Discovery (OD) ( ) module in frame $\mathbf{F}_t$. Relevance scores $c_{i,t}$ are computed and $\mathcal{M}_{ego}$ is updated.

means of a visual object tracker [46, 52]. Alongside each $\mathbf{b}_{i,t}$, the tracker outputs confidence scores $s_{i,t}$ and only bounding boxes with $s_{i,t} > \lambda_{ot}$ are retained. After being filtered by the OT, the remaining bounding boxes and their associated frames are written to memory by calling $\mathcal{M}_{ego}.\texttt{write}\left([t, \mathbf{b}_{i,t}, \mathbf{F}_{i,t}, c_{i,t}]\right)$. If no objects are stored in $\mathcal{M}_{ego}$, i.e. when $t = 0$ or until the new objects are inserted, the execution of the OT module is skipped. See the supplementary material for more details about the implementation of the OT module.

Object Discovery Module. The OD module is responsible for finding objects of interest in frame $\mathbf{F}_t$ which could be later queried by the user. OD is implemented as an object detector [49] which detects a set of B bounding boxes $\mathbf{b}_{k,t}$ of objects in the frame, coupled with confidence scores $s_{k,t}$, with $1 \leq k \leq B$. After detecting the objects of interest, the OD module calls $\mathcal{M}_{ego}.\texttt{read}(t)$ to retrieve the bounding boxes of all object instances already present in memory at time t. These are objects which may already have been tracked by the OT module at time t from object instances at time $t-1$. The detected boxes are first filtered based on $s_{k,t} > \lambda_{od}$. Those that do not have an IoU $> \lambda_{iou}$ with any of the objects retrieved from the memory, are treated as localizations of new object instances. Filtered $\mathbf{b}_{k,t}$ together with the associated frame $\mathbf{F}_t$, are then stored in memory by calling $\mathcal{M}_{ego}.\texttt{write}\left([t, \mathbf{b}_{k,t}, \mathbf{F}_t, c_{k,t}]\right)$, while the others are discarded. See the supplementary material for details.

Retrieval Relevance. To enhance the efficiency of the retrieval stage, each call to the `.write()` method by OT and OD is preceded by a function that computes relevance scores $c_{i,t}$. These scores identify object patches $\phi_{i,t}$, ex-

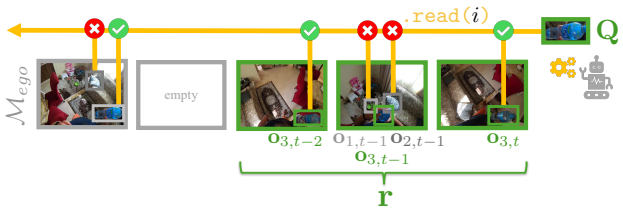

Figure 4. **Visual query localization by memory retrieval.** When the user provides a visual query Q, the Query Retrieval and Localization algorithm ( ) is triggered. This algorithm compares the representation of Q with the representations of each object $\mathbf{O}_i$ in $\mathcal{M}_{ego}$. The sequence of contiguous bounding-boxes corresponding to the best-matched ( ) object and the associated RGB frames are retained as the visual response track r for Q, while all other objects ( ) are discarded.

tracted from $\mathbf{b}_{i,t}$ in $\mathbf{F}_t$, that contain sufficiently distinctive features for accurate instance recognition. We employ a binary classifier to assess whether each patch $\phi_{i,t}$ meets the quality threshold for effective retrieval (see Supp. material for details). If the classifier determines that a patch is suitable, $c_{i,t} = 1$, otherwise, $c_{i,t} = 0$.

4.3. Query Retrieval and Localization (QRL)

The Query Retrieval and Localization (QRL) algorithm is responsible for retrieving the visual response track $\mathbf{r}$ of a visual query $\mathbf{Q}$ within the memory $\mathcal{M}_{ego}$. This algorithm takes $\mathbf{Q}$ as input and outputs a sequence of contiguous bounding boxes $\mathbf{b}_{i,t}$ and frames $\mathbf{F}_t$ for the object instance $\mathbf{O}_i$ in $\mathcal{M}_{ego}$ that visually matches $\mathbf{Q}$. The process begins by extracting the query feature representation $\Phi(\mathbf{Q})$ using a feature extraction function $\Phi(\cdot)$. Such representation is hence compared against features $\Phi(\phi_{i,t})$ of object states $\mathbf{o}_{i,t}$ with $c_{i,t} = 1$ using a feature similarity function $\Psi(\Phi(\mathbf{Q}), \Phi(\phi_{i,t})) \in [0, 1]$. We average similarity scores related to multiple boxes $\mathbf{o}_{i,t}$ of the same object i into a single score r_i. If the maximum score $r_{\bar{i}}$ across all objects exceeds the threshold λ_{ret}, the object $\mathbf{O}_{\bar{i}}$ is considered a match for query $\mathbf{Q}$. We hence return the most recent sequence of contiguous bounding boxes and frames from the spatio-temporal history of $\mathbf{O}_{\bar{i}}$ as the visual response track $\mathbf{r}$. Note that this process relies exclusively on $\mathcal{M}_{ego}$, and it does not require to access $\mathcal{V}_{ego}$. Figure 4 provides an illustration of the process performed by QRL.

5. Experimental Settings

Datasets. We evaluate ESOM on the Ego4D Episodic Memory VQ2D benchmark [22], including 433 hours of video from 54 scenarios, featuring $22K$ visual queries spanning $3K$ object categories. At present time, this is the only publicly available benchmark for Visual Query Localization. To train components, we also use EgoTracks [46], an egocentric video dataset sourced from Ego4D and aligned to the episodic memory benchmark, and EgoObjects [57], a large-scale egocentric object detection dataset.

Implementation Details. We tested different configurations of the proposed ESOM framework. In OMP, for the OT component, we used EgoSTARK [46], a STARK instance [52] fine-tuned on EgoTracks, as well as the MASA tracker [27]. To evaluate optimal tracking performance, we also considered ground-truth annotations from EgoTracks. We refer to these configurations as $\text{OT}_{egostark}$, OT_{masa}, OT_{oracle}. For the OD module, we considered the off-the-shelf open-world detector GroundingDino [30], an instance of YOLOv10 [49] trained on curated versions of EgoTracks [46] and EgoObjects [57], and ground truth object detections from EgoTracks. These are denoted as OD_{gdino}, OD_{yolo}, OD_{oracle}. In QRL, we implemented feature extraction $\Phi(\cdot)$ and similarity $\Psi(\cdot)$ using two configurations: Siam-RCNN with a classification head [48] or DINOv2 with cosine similarity [21, 34], referred to as $\text{QRL}_{siamrcnn}$ and QRL_{dino}. We obtained best results with $\text{OT}_{egostark}$, OD_{yolo} and $\text{QRL}_{siamrcnn}$, which identify the default configuration of ESOM where not otherwise specified. See the supplementary material for more implementation details.

Evaluation Protocol and Measures. Each video clip $\mathcal{V}_{ego}$ in the evaluation dataset is fed to the OMP algorithm to construct the object memory $\mathcal{M}_{ego}$. Each visual query $\mathbf{Q}$ provided with the clip is then processed using the QRL algorithm, producing a visual response track $\mathbf{r}$, which we subsequently compare to the annotated ground-truth response track of bounding-boxes to compute query localization accuracy. We use the standard VQL evaluation metrics [22]: $\%\text{tAP}_{25} \uparrow$, measures temporal alignment with the ground-truth at a 0.25 IoU threshold; $\%\text{stAP}_{25} \uparrow$, assesses spatio-temporal precision a 0.25 spatio-temporal IoU threshold; Success (%Succ $\uparrow$) calculates the proportion of predictions with a minimum 0.05 spatio-temporal IoU with the ground-truth response track. All metrics are reported in the range [0, 100]. In addition to these standard measures, we evaluate efficiency in terms of storage size (Size $\downarrow$) in multiples of bytes (MB, GB) needed to store information that enables query localization, and retrieval time (Time $\downarrow$) in seconds (s) or minutes (m) that is required to localize a visual query.

6. Results

6.1. Online Visual Query 2D (OVQ2D) Benchmark

Table 1 benchmarks different configurations of ESOM (3-6) against two baselines (1-2) that apply our QRL method on object memory representations recently proposed for tasks other than Online Episodic Memory. AMEGO [21] is designed for interaction-based egocentric video understanding and focuses on objects that the camera wearer has in-

	Method	OD	OT	%tAP$_{25}$ ↑	%stAP$_{25}$ ↑	%Succ ↑	Size ↓	Time ↓
1	AMEGO [21]	OD$_{hoi-det}$	OT$_{egostark}$	0.0	0.0	0.67	531.6 MB	1.5 s
2	VideoAgent [15]	OD$_{rt-detr}$	OT$_{bytetrack}$	0.02	0.1	3.03	13.7 GB	12.1 s
3	ESOM	OD$_{gdino}$	OT$_{masa}$	0.002	0.001	0.70	1.3 GB	2.5 s
4	ESOM	OD$_{gdino}$	OT$_{egostark}$	0.04	0.002	0.72	8.8 GB	24.1 s
5	ESOM	OD$_{yolo}$	OT$_{masa}$	**0.1**	0.1	3.75	**383.6 MB**	**0.7 s**
6	ESOM	OD$_{yolo}$	OT$_{egostark}$	0.03	**0.2**	**4.02**	1.7 GB	3.1 s
7	ESOM	OD$_{oracle}$	OT$_{egostark}$	18.3	5.1	31.91	591.4 MB	0.5 s
8	ESOM	OD$_{yolo}$	OT$_{oracle}$	21.0	19.4	40.55	322.5 MB	0.6 s
9	ESOM	OD$_{oracle}$	OT$_{oracle}$	73.2	68.1	81.92	507.4 MB	0.5 s

Table 1. **Comparison of different configurations of ESOM for OVQ2D with respect to other online approaches.** ESOM outperforms competitors overall, with OD$_{yolo}$-OT$_{egostark}$ being the best configuration according to success rate and OD$_{yolo}$-OT$_{masa}$ producing much faster inference and more compact memory sacrificing some performance. Grayed rows (7-9) report ESOM performance in oracle configurations. (Best results are highlighted in **bold**.)

teracted with by hand, whereas VideoAgent [15] tackles video question answering and focuses on generic objects. Since AMEGO and VideoAgent were not originally designed for the OVQ2D task, their baselines in Table 1 are adapted by integrating their respective object detection and tracking components—HOI-detector [43] and EgoStark for AMEGO, RT-DETR [56] and ByteTrack [54] for VideoAgent—with our memory and retrieval modules. Alongside real-world configurations, in (7-9), we also report baselines using oracular detection (7), object tracking (8) and both (9). Best results are obtained by the proposed ESOM when state-of-the-art detector (OD$_{yolo}$) and tracker (OT$_{egostark}$) are considered. Besides being competitive in terms of query retrieval accuracy (see columns 4-6), this configuration brings a fast retrieval of 3.1 s and a compact memory of 1.7 GB. Among competitors, VideoAgent achieves a lower success rate (3.03 vs 4.02) with much larger memory 13.7 vs 1.7 GB and slower retrieval time (12.5 vs 3.1 s), while AMEGO, focusing only on hand-interacted objects, achieves a very low success rate of 0.67. The choice of detector and tracker greatly affects the performance of ESOM. Indeed, using the generic object detector OD$_{gdino}$, rather than our optimized OD$_{yolo}$ leads to very limited success rates, regardless of the tracker. OT$_{masa}$ and OT$_{egostark}$ also lead to different results, with OT$_{egostark}$ bringing the best performance, while OT$_{masa}$ allowing to obtain faster retrieval and a more compact memory sacrificing some retrieval accuracy. It is worth noting that the OVQ2D benchmark is very challenging for current approaches, with the best methods achieving a success rate of just 4.02 and very low object localization scores overall. The oracular results in rows 7-9 show that this is mainly due to the limited effectiveness of current object detection and tracking approaches in real-world egocentric videos. Indeed, pairing OT$_{egostark}$ with ground truth detections (row 7) brings to a success rate of 31.91, pairing a OD$_{yolo}$ object detection with an oracular tracker leads to a success rate of 40.55, while using both oracular detection and tracking leads to 81.92. Overall, these results highlight the potential of ESOM to tackle OVQ2D, but also highlight that efforts should be devoted to improve object detection and tracking. While prior studies highlight challenges in egocentric object detection [57] and tracking [11–13, 46], algorithms are typically evaluated in isolation [46, 57], which we argue may misrepresent their real-world reliability. Beyond OVQ2D, this benchmark, including its oracular baselines, can serve as a principled testbed to support the development and evaluation of object detection and tracking in real-world, downstream settings, rather than in isolation.

Method	%Succ ↑	Size ↓	Time ↓
Offline Methods (VQ2D)			
SiamRCNN + KYS [22]	39.80	12.1 GB	8.3 m
STARK [52]	18.70	12.1 GB	45 s
SiamRCNN [48]	43.24	12.1 GB	8.3 m
CocoFormer [51]	48.37	12.1 GB	8.3 m
VQLoC [26]	55.89	12.1 GB	41 s
PRVQL [14]	57.93	12.1 GB	50 s
ESOM (OVQ2D)			
ESOM - OD$_{oracle}$ - OT$_{oracle}$	81.92	507.4 MB	0.5 s

Table 2. **Comparison of ESOM in OVQ2D versus offline VQ2D methods.** Note that VQ2D is much less challenging than OVQ2D, hence success rate is not directly comparable and these results are reported to discuss the differences between the two approaches. ESOM offers significant savings in memory storage and enhances the efficiency of query localization.

6.2. Comparison with Offline VQ2D

Table 2 shows state-of-the-art offline VQ2D methods. It is important to note that offline VQ2D and our online formulation are not directly comparable, OVQ2D is a much harder task in which methods do not know have access to the queried object in advance; hence these results are reported mainly to discuss the differences between the two approaches. Current offline methods require storing the en-

OMP Configuration OD	OT	QRL	%tAP$_{25}$ ↑	%stAP$_{25}$ ↑	%Succ ↑	Time ↓
OD$_{oracle}$	OT$_{oracle}$	QRL$_{siamrcnn}$	**73.2**	**68.1**	**81.92**	**0.5 s**
		QRL$_{dino}$	64.4	57.7	76.00	0.6 s
OD$_{yolo}$	OT$_{oracle}$	QRL$_{siamrcnn}$	**21.0**	**19.4**	**40.55**	**0.4 s**
		QRL$_{dino}$	20.6	12.1	35.53	0.4 s
OD$_{oracle}$	OT$_{egostark}$	QRL$_{siamrcnn}$	**18.3**	**5.1**	**31.91**	**0.5 s**
		QRL$_{dino}$	**18.3**	**5.1**	30.68	0.5 s
OD$_{yolo}$	OT$_{egostark}$	QRL$_{siamrcnn}$	**0.02**	**0.2**	**4.02**	**3.1 s**
		QRL$_{dino}$	**0.02**	**0.2**	3.79	3.4 s

Table 3. **Effect of the QRL algorithm on performance.** Performance of QRL methods across various OMP configuations. QRL$_{siamrcnn}$ consistently achieves the highest accuracy, making it the best-performing model for query retrieval in OVQ2D. (Best results are highlighted in **bold**).

OMP Configuration OD	OT	Retr. Rel.	%tAP$_{25}$ ↑	%stAP$_{25}$ ↑	%Succ ↑	Time ↓
OD$_{oracle}$	OT$_{oracle}$	✓	**73.2**	**68.1**	**81.92**	**0.5 s**
		✗	71.5	67.4	81.37	4.6 s
OD$_{yolo}$	OT$_{oracle}$	✓	**21.0**	**19.4**	**40.55**	**0.4 s**
		✗	18.3	18.1	**40.55**	5.3 s
OD$_{oracle}$	OT$_{egostark}$	✓	**18.3**	**5.1**	**31.91**	**0.5 s**
		✗	17.4	5.0	29.78	24.2 s
OD$_{yolo}$	OT$_{egostark}$	✓	0.02	**0.2**	**4.02**	**3.1 s**
		✗	**0.03**	0.1	2.68	24.8 s

Table 4. **Ablating the use of the retrieval relevance over different OMP configurations.** Using relevant visual representations for retrieval (Retr. Rel.) improves time and accuracy of QRL. (Best results in **bold**).

tire video sequence, resulting in significant memory usage. ESOM reduces memory needs by up 7× (1.7 vs 12.1GB) and increases inference speed by 13× when compared to VQLoC (3.1 vs 41 s). Memory and computational savings are even more significant when considering the oracular configuration, reported for reference, with 24× less memory and 80× increased inference speed. Interestingly, ESOM is competitive even in terms of success rate in oracular settings (81.92 vs 55.89 of VQLoC), which highlights the potential of OVQ2D and ESOM when tracking and detection are solved.

6.3. Detailed Analysis of ESOM and Ablations

Effect of QRL algorithm on performance. Table 3 compares the performance of the QRL algorithm under the two considered implementations. QRL$_{siamrcnn}$ consistently performs best at matching a visual query with a memory of stored representations. Across all memory constructions produced by various OMP configurations, QRL$_{siamrcnn}$ demonstrates superior query localization scores, emphasizing its strength and adaptability for the OVQ2D task.

Filtering visual representations helps retrieval accuracy and time. We evaluated the impact of computing relevance scores $c_{i,t}$ via the binary classifier before the `.write()` step on QRL accuracy and speed (Table 4). Without relevance filtering, all stored object representations are considered during retrieval, increasing computation time as more similarities must be evaluated. Filtering also improves the downstream OVQ2D performance by discarding suboptimal object representations.

Effect of OD and OT modules. Table 5 reports ESOM's performance across different OD and OT configurations, including module-specific and VQL task metrics: AP ↑ and Rec ↑ for object detection, HOTA ↑ and AssA ↑ for object tracking [32], and %(s)tAP$_{25}$, Succ., Size, and Time for downstream OVQ2D.

The OD module has a noticeable impact on performance, as weaker detectors might miss important queries and add objects into memory that are unlikely to be queried later. OD$_{gdino}$, designed for open-world detection, has low recall and AP for annotated queries, leading to weak retrieval even with OT$_{oracle}$. In contrast, OD$_{yolo}$, trained to detect queried objects, enables better memorization and improves QRL accuracy. Changes in the detector impact memory size and processing time; OD$_{gdino}$ detects more objects, expanding memory and increasing query localization time.

OT modules have a more significant impact on performance. replacing the oracular tracker OT$_{oracle}$ with OT$_{egostark}$ while keeping OD$_{oracle}$ brings Succ. down from 81.92 to 31.91. We observe an inverse correlation between downstream and general tracking performance. For instance, OT$_{masa}$ achieves the highest HOTA score (50.1 vs. 21.7), yet yields significantly lower Succ. (9.38 vs. 31.91). This stems from HOTA being the geometric mean of DetA (detection accuracy within individual frames) and AssA (association accuracy across time) [32]. In our evaluations, DetA tends to be consistently lower than AssA — for example, OD$_{oracle}$–OT$_{egostark}$ shows 13.7 vs. 34.7, and OD$_{yolo}$–OT$_{egostark}$ yields 2.5 vs. 15.1—ultimately reducing overall HOTA scores. OVQ2D requires higher AssA than DetA, as accurate object association is essential for constructing coherent and temporally consistent response tracks in $\mathcal{M}_{ego}$. Incorrect associations lead to fragmented or duplicated tracklets, which DetA penalizes. OD$_{oracle}$–OT$_{masa}$ stands out as an exception, relying on ground truth detections and thus isolating the role of association, which explains its strong HOTA despite limited downstream success. Being explicitly optimized for egovision, OT$_{egostark}$ leads to the best downstream performance.

We consider a fixed memory of 12.1 GB to store a 5-minutes clip as required in the EGO4D benchmark. Longer clips would take more space.

All detections are merged into a single "object" class.

OMP Config. OD	AP ↑	Rec ↑	OMP Config. OT	HOTA ↑	AssA ↑	%tAP$_{25}$ ↑	%stAP$_{25}$ ↑	%Succ ↑	Size ↓	Time ↓
OD$_{oracle}$	100.0	100.0	OT$_{oracle}$	100.0	100.0	73.2	68.1	81.92	507.4 MB	0.5 s
			OT$_{masa}$	**50.1**	25.2	2.8	1.6	9.38	1.3 GB	7.2 s
			OT$_{egostark}$	21.7	**34.7**	**18.3**	**5.1**	**31.91**	**591.4 MB**	**0.5 s**
OD$_{yolo}$	13.6	30.7	OT$_{oracle}$	84.8	94.3	21.0	19.4	40.55	322.5 MB	0.4 s
			OT$_{masa}$	**6.6**	11.6	**0.1**	**0.1**	3.75	**383.6 MB**	**0.7 s**
			OT$_{egostark}$	6.1	**15.1**	0.03	0.2	**4.02**	1.7 GB	3.1 s
OD$_{gdino}$	4.5	25.7	OT$_{oracle}$	87.5	94.3	15.7	12.2	33.33	355.1 MB	0.5 s
			OT$_{masa}$	3.6	10.5	0.0	0.0	0.70	**1.3 GB**	**2.5 s**
			OT$_{egostark}$	**5.7**	**15.1**	**0.04**	0.0	**0.72**	8.7 GB	24.1 s

Table 5. **Effects of OD and OT modules on performance.** Different OD and OT model implementations determine different behaviors of the OMP algorithm, resulting in different memory representations. This, in turn leads to different QRL results. Performance of methods with oracular object tracking is reported in gray. (Best results are in **bold**).

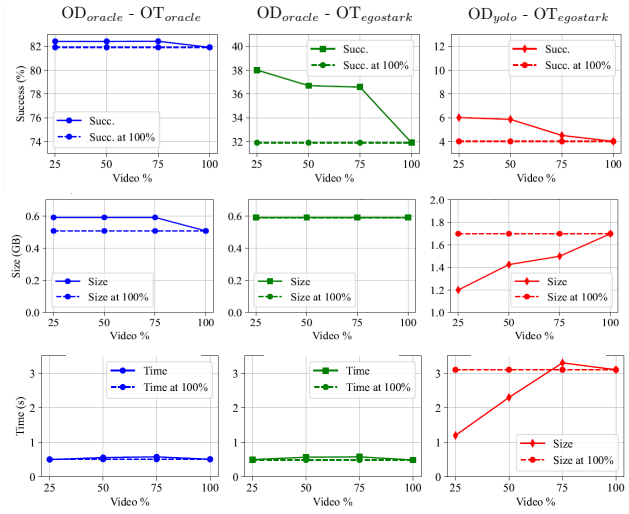

Figure 5. **ESOM scales well while increasing memory size.** Plots show how success score, storage space, and retrieval time change when building memory from progressively processed video segments. Results for three different OMP configurations.

Behavior of ESOM with different amounts of memory. Figure 5 shows retrieval performance, storage space, and retrieval time as 25%, 50%, 75%, and 100% of videos are observed with different OMP configurations. QRL performance declines slightly with video length and remains steady with oracular OD and OT, while minor drops occur with OD$_{oracle}$ and OD$_{egostark}$, underscoring ESOM's scalability. Storage and retrieval time generally increase over time; however, memory size remains stable with OD$_{oracle}$ since most objects appear within the first 25% of the video. OD$_{yolo}$ - OT$_{egostark}$ significantly impacts storage and retrieval due to duplicate objects from weaker tracking.

Qualitative Results. Figure 6 shows a qualitative example of QRL retrieving and localizing a visual query within two memory representations.

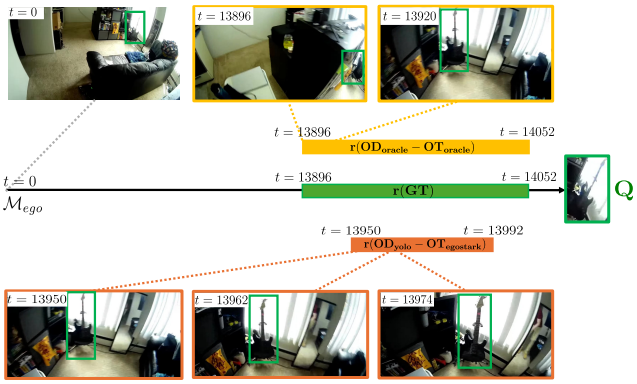

Figure 6. **Qualitative example.** Successful retrieval of query Q from $\mathcal{M}_{ego}$ using the oracle OMP algorithm (OD$_{oracle}$ - OT$_{oracle}$) and the real-world OMP (OD$_{yolo}$ - OT$_{egostark}$). Sample frames from each track are shown with corresponding bounding boxes whose patches $\phi_{i,t}$ were matched during retrieval. The frame $t = 0$ corresponds to the moment of the object's discovery. The green bar visualizes the temporal extent of the ground-truth track, with timestamps indicating its duration.

7. Conclusions

We introduced the novel task of Online Visual Queries 2D (OVQ2D), where a video is processed in a streaming fashion and an object query must be localized without access to the original video. To address it, we proposed ESOM (Egocentric Stream Object Memory), an online approach that detects, tracks, and stores essential objects to support efficient visual query retrieval and localization. ESOM significantly optimizes both memory and computational efficiency, reducing memory usage by up to 7×, shortening retrieval times from minutes to a few seconds. On the Ego4D dataset ESOM outperforms other online methods and shows further gains when object detection and tracking components are improved. Besides establishing a benchmark for OVQ2D, we also contribute a principled evaluation, based on a real downstream task, for object detection and tracking algorithms for egovision applications.

Acknowledgments. Progetto PRIN 2022 PNRR - "Tracking in Egovision for Applied Memory (TEAM)" Codice P20225MSER_001. Codici CUP G53D23006680001 (Università di Udine) e E53D23016240001 (Università di Catania). This research has been funded by the European Union, NextGenerationEU – PNRR M4 C2 I1.1 RS Micheloni. MD received funding from the European Union's Horizon Europe research and innovation programme under the Marie Skłodowska-Curie grant agreement n. 101151834 PRINNEVOT (CUP G23C24000910006).

References

[1] Nir Aharon, Roy Orfaig, and Ben-Zion Bobrovsky. Botsort: Robust associations multi-pedestrian tracking. *arXiv preprint arXiv:2206.14651*, 2022. 3

[2] Joungbin An, Hyolim Kang, Su Ho Han, Ming-Hsuan Yang, and Seon Joo Kim. Miniroad: Minimal rnn framework for online action detection. In *ICCV*, 2023. 3

[3] Leonard Bärmann and Alex Waibel. Where did i leave my keys?-episodic-memory-based question answering on egocentric videos. In *CVPRW*, 2022. 3

[4] Gedas Bertasius, Heng Wang, and Lorenzo Torresani. Is space-time attention all you need for video understanding? In *ICML*, 2021. 3

[5] Goutam Bhat, Martin Danelljan, Luc Van Gool, and Radu Timofte. Know your surroundings: Exploiting scene information for object tracking. In *ECCV*, 2020. 2

[6] João Carreira and Andrew Zisserman. Quo vadis, action recognition? a new model and the kinetics dataset. In *CVPR*, 2017. 3

[7] Guo Chen, Sen Xing, Zhe Chen, Yi Wang, Kunchang Li, Yizhuo Li, Yi Liu, Jiahao Wang, Yin-Dong Zheng, Bingkun Huang, et al. Internvideo-ego4d: A pack of champion solutions to ego4d challenges. *arXiv preprint arXiv:2211.09529*, 2022. 1, 2

[8] Tianheng Cheng, Lin Song, Yixiao Ge, Wenyu Liu, Xinggang Wang, and Ying Shan. Yolo-world: Real-time open-vocabulary object detection. In *CVPR*, 2024. 3, 4

[9] Dima Damen, Hazel Doughty, Giovanni Maria Farinella, Antonino Furnari, Evangelos Kazakos, Jian Ma, Davide Moltisanti, Jonathan Munro, Toby Perrett, Will Price, and Michael Wray. Rescaling egocentric vision: Collection, pipeline and challenges for epic-kitchens-100. *IJCV*, 2022. 3

[10] Roeland De Geest, Efstratios Gavves, Amir Ghodrati, Zhenyang Li, Cees Snoek, and Tinne Tuytelaars. Online action detection. In *ECCV*, 2016. 3

[11] Matteo Dunnhofer, Antonino Furnari, Giovanni Maria Farinella, and Christian Micheloni. Is first person vision challenging for object tracking? In *ICCVW*, 2021. 6, 2

[12] Matteo Dunnhofer, Antonino Furnari, Giovanni Maria Farinella, and Christian Micheloni. Visual object tracking in first person vision. *IJCV*, 2023. 2

[13] Matteo Dunnhofer, Zaira Manigrasso, and Christian Micheloni. Is tracking really more challenging in egocentric first person vision? In *ICCV*, 2025. 6

[14] Bing Fan, Yunhe Feng, Yapeng Tian, Yuewei Lin, Yan Huang, and Heng Fan. Prvql: Progressive knowledge-guided refinement for robust egocentric visual query localization. *arXiv preprint arXiv:2502.07707*, 2025. 6

[15] Yue Fan, Xiaojian Ma, Rujie Wu, Yuntao Du, Jiaqi Li, Zhi Gao, and Qing Li. Videoagent: A memory-augmented multimodal agent for video understanding. In *ECCV*, 2024. 3, 6, 1, 4

[16] Christoph Feichtenhofer, Haoqi Fan, Jitendra Malik, and Kaiming He. Slowfast networks for video recognition. In *ICCV*, 2019. 3

[17] Yisen Feng, Haoyu Zhang, Yuquan Xie, Zaijing Li, Meng Liu, and Liqiang Nie. Objectnlq@ ego4d episodic memory challenge 2024. *arXiv preprint arXiv:2406.15778*, 2024. 1

[18] Antonino Furnari and Giovanni Farinella. Rolling-unrolling lstms for action anticipation from first-person video. *IEEE TPAMI*, 2021. 3

[19] Jiyang Gao, Zhenheng Yang, and Ram Nevatia. RED: Reinforced encoder-decoder networks for action anticipation. In *BMVC*, 2017.

[20] Rohit Girdhar and Kristen Grauman. Anticipative video transformer. In *ICCV*, 2021. 3

[21] Gabriele Goletto, Tushar Nagarajan, Giuseppe Averta, and Dima Damen. Amego: Active memory from long egocentric videos. In *ECCV*, 2024. 3, 5, 6, 1

[22] Kristen Grauman et al. Ego4d: Around the World in 3,000 Hours of Egocentric Video. In *CVPR*, 2022. 1, 2, 3, 5, 6

[23] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *CVPR*, 2016. 1

[24] Minh Hoai and Fernando De la Torre. Max-margin early event detectors. *IJCV*, 2014. 3

[25] Yixuan Huang, Jialin Yuan, Chanho Kim, Pupul Pradhan, Bryan Chen, Li Fuxin, and Tucker Hermans. Out of sight, still in mind: Reasoning and planning about unobserved objects with video tracking enabled memory models. *arXiv preprint arXiv:2309.15278*, 2023. 2

[26] Hanwen Jiang, Santhosh Kumar Ramakrishnan, and Kristen Grauman. Single-stage visual query localization in egocentric videos. In *NeurIPS*, 2023. 1, 2, 6

[27] Siyuan Li, Lei Ke, Martin Danelljan, Luigi Piccinelli, Mattia Segu, Luc Van Gool, and Fisher Yu. Matching anything by segmenting anything. In *CVPR*, 2024. 5, 2

[28] Tsung-Yi Lin, Piotr Dollár, Ross Girshick, Kaiming He, Bharath Hariharan, and Serge Belongie. Feature pyramid networks for object detection. In *CVPR*, 2017. 5

[29] Naiyuan Liu, Xiaohan Wang, Xiaobo Li, Yi Yang, and Yueting Zhuang. Reler@ zju-alibaba submission to the ego4d natural language queries challenge 2022. *arXiv preprint arXiv:2207.00383*, 2022. 1

[30] Shilong Liu, Zhaoyang Zeng, Tianhe Ren, Feng Li, Hao Zhang, Jie Yang, Qing Jiang, Chunyuan Li, Jianwei Yang, Hang Su, et al. Grounding dino: Marrying dino with grounded pre-training for open-set object detection. *arXiv preprint arXiv:2303.05499*, 2023. 5, 3, 4

[31] Shuming Liu, Chen-Lin Zhang, Chen Zhao, and Bernard Ghanem. End-to-end temporal action detection with 1b parameters across 1000 frames. In *CVPR*, 2024. 3

[32] Jonathon Luiten, Aljosa Osep, Patrick Dendorfer, Philip Torr, Andreas Geiger, Laura Leal-Taixé, and Bastian Leibe. Hota: A higher order metric for evaluating multi-object tracking. *IJCV*, 2021. 7, 2

[33] Jinjie Mai, Abdullah Hamdi, Silvio Giancola, Chen Zhao, and Bernard Ghanem. Egoloc: Revisiting 3d object localization from egocentric videos with visual queries. In *ICCV*, 2023. 1

[34] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, et al. Dinov2: Learning robust visual features without supervision. *arXiv preprint arXiv:2304.07193*, 2023. 5

[35] Baoqi Pei, Guo Chen, Jilan Xu, Yuping He, Yicheng Liu, Kanghua Pan, Yifei Huang, Yali Wang, Tong Lu, Limin Wang, et al. Egovideo: Exploring egocentric foundation model and downstream adaptation. *arXiv preprint arXiv:2406.18070*, 2024. 1, 2

[36] Chiara Plizzari, Gabriele Goletto, Antonino Furnari, Siddhant Bansal, Francesco Ragusa, Giovanni Maria Farinella, Dima Damen, and Tatiana Tommasi. An outlook into the future of egocentric vision. *IJCV*, 2024. 1

[37] Chiara Plizzari, Shubham Goel, Toby Perrett, Jacob Chalk, Angjoo Kanazawa, and Dima Damen. Spatial cognition from egocentric video: Out of sight, not out of mind. In *3DV*, 2025. 3

[38] Santhosh Kumar Ramakrishnan, Ziad Al-Halah, and Kristen Grauman. Naq: Leveraging narrations as queries to supervise episodic memory. In *CVPR*, 2023. 1

[39] Santhosh Kumar Ramakrishnan, Ziad Al-Halah, and Kristen Grauman. Spotem: efficient video search for episodic memory. In *ICML*, 2023. 1

[40] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster r-cnn: Towards real-time object detection with region proposal networks. *IEEE TPAMI*, 2016. 3

[41] Ivan Rodin, Antonino Furnari, Dimitrios Mavroedis, and Giovanni Maria Farinella. Predicting the future from first person (egocentric) vision: A survey. *CVIU*, 2021. 3

[42] Mohammad Sadegh Aliakbarian, Fatemeh Sadat Saleh, Mathieu Salzmann, Basura Fernando, Lars Petersson, and Lars Andersson. Encouraging lstms to anticipate actions very early. In *ICCV*, 2017. 3

[43] Dandan Shan, Jiaqi Geng, Michelle Shu, and David F Fouhey. Understanding human hands in contact at internet scale. In *CVPR*, 2020. 6, 3, 4

[44] Kiran Somasundaram, Jing Dong, Huixuan Tang, Julian Straub, Mingfei Yan, Michael Goesele, Jakob Julian Engel, Renzo De Nardi, and Richard Newcombe. Project aria: A new tool for egocentric multi-modal ai research. *arXiv preprint arXiv:2308.13561*, 2023. 2

[45] Alexandros Stergiou and Dima Damen. The wisdom of crowds: Temporal progressive attention for early action prediction. In *CVPR*, 2023. 3

[46] Hao Tang, Kevin J Liang, Kristen Grauman, Matt Feiszli, and Weiyao Wang. Egotracks: a long-term egocentric visual object tracking dataset. In *NeurIPS*, 2023. 4, 5, 6, 1, 2, 3

[47] Endel Tulving. Episodic memory: From mind to brain. *Annual review of Psychology*, 2002. 1

[48] Paul Voigtlaender, Jonathon Luiten, Philip HS Torr, and Bastian Leibe. Siam r-cnn: Visual tracking by re-detection. In *CVPR*, 2020. 2, 5, 6, 1

[49] A Wang, H Chen, L Liu, K Chen, Z Lin, J Han, and G Ding. Yolov10: Real-time end-to-end object detection. arxiv 2024. *arXiv preprint arXiv:2405.14458*, 2024. 4, 5, 3

[50] Mengmeng Xu, Cheng-Yang Fu, Yanghao Li, Bernard Ghanem, Juan-Manuel Perez-Rua, and Tao Xiang. Negative frames matter in egocentric visual query 2d localization. *arXiv preprint arXiv:2208.01949*, 2022. 1, 2

[51] Mengmeng Xu, Yanghao Li, Cheng-Yang Fu, Bernard Ghanem, Tao Xiang, and Juan-Manuel Pérez-Rúa. Where is my wallet? modeling object proposal sets for egocentric visual query localization. In *CVPR*, 2023. 1, 2, 6

[52] Bin Yan, Houwen Peng, Jianlong Fu, Dong Wang, and Huchuan Lu. Learning spatio-temporal transformer for visual tracking. In *ICCV*, 2021. 4, 5, 6, 2

[53] Chen-Lin Zhang, Jianxin Wu, and Yin Li. Actionformer: Localizing moments of actions with transformers. In *ECCV*, 2022. 3

[54] Yifu Zhang, Peize Sun, Yi Jiang, Dongdong Yu, Fucheng Weng, Zehuan Yuan, Ping Luo, Wenyu Liu, and Xinggang Wang. Bytetrack: Multi-object tracking by associating every detection box. In *ECCV*, 2022. 6

[55] Yue Zhao and Philipp Krähenbühl. Real-time online video detection with temporal smoothing transformers. In *ECCV*, 2022. 3

[56] Yian Zhao, Wenyu Lv, Shangliang Xu, Jinman Wei, Guanzhong Wang, Qingqing Dang, Yi Liu, and Jie Chen. Detrs beat yolos on real-time object detection. In *CVPR*, 2024. 6

[57] Chenchen Zhu, Fanyi Xiao, Andrés Alvarado, Yasmine Babaei, Jiabo Hu, Hichem El-Mohri, Sean Culatana, Roshan Sumbaly, and Zhicheng Yan. Egoobjects: A large-scale egocentric dataset for fine-grained object understanding. In *ICCV*, 2023. 5, 6, 3

Online Episodic Memory Visual Query Localization with Egocentric Streaming Object Memory

Supplementary Material

This document reports implementation details on the proposed ESOM methodology, along with additional results and discussion. Specifically, Section A explains the ESOM architecture by providing details of the Object Memory representation (sub-section A.1), the Object Tracking (OT) module (sub-section A.2.2), on the Object Discovery (OD) module (Section A.2.2), and on the Query Retrieval and Localization (QRL) algorithm (sub-section A.3). Section B reports details on the implementation of the baselines methods on AMEGO [21] and VideoAgent [15]. Section C reports information on the evaluation dataset and Section D reports additional and qualitative results.

A. Implementation Details of ESOM

A.1. Object Memory

Visual representation filter. To improve retrieval time and accuracy, the `.write()` operation incorporates a filtering mechanism to determine whether the visual content within $\mathbf{b}_{i,t}$, $\phi_{i,t}$, is relevant for the retrieval process. This filtering process relies on a binary classification function that assigns a label $c_{i,t}$ to each visual representation $\phi_{i,t}$, assessing its quality for effective query retrieval.
The design of $\mathcal{M}_{ego}$, which includes information about the quality of data for retrieval, was guided by experimental results. Specifically, we tested various filtering approaches to determine their effectiveness.

- MR1* (*design as reported in Section 4.1 of the main paper*): for each object $\mathbf{O}_i$, the filter assigns $c_{i,t} = 1$ at the moment of object discovery, as well as to two bounding-boxes $\mathbf{b}_{i,t}$ and their associated frames $\mathbf{F}_t$ from the most recent response track. These bounding-box and frame pairs are selected as the first and last valid patches identified by the classifier within the latest sequence of bounding boxes. For all other frames in temporal locations not mentioned, the filter set $c_{i,t} = 0$.
- MR2: for each object $\mathbf{O}_i$, the filter assigns $c_{i,t} = 1$ at the moment of object discovery, as well as to all the bounding-boxes $\mathbf{b}_{i,t}$ and their associated frames $\mathbf{F}_t$ from the most recent response track, considered useful by the classifier. This implementation was conducted to assess the impact of exploiting more object visual information for QRL.
- MR3: for each object $\mathbf{O}_i$, the filter assigns $c_{i,t} = 1$ at the moment of object discovery. For all other frames in temporal locations not mentioned, the filter set $c_{i,t} = 0$. This implementation was conducted to assess the impact of exploiting less object visual information for QRL, thus reducing the retrieval time of $\mathcal{M}_{ego}$.
- MR4: for each object $\mathbf{O}_i$, the filter assigns $c_{i,t} = 1$ to two bounding-boxes $\mathbf{b}_{i,t}$ and their associated frames $\mathbf{F}_t$ from the most recent response track. These bounding-box and frame pairs are selected as the first and last valid patches identified by the classifier within the latest sequence of bounding boxes. For all other frames in temporal locations not mentioned, the filter set $c_{i,t} = 0$. This implementation was conducted to assess the impact of considering the object at the moment of discovery in retrieval performances.

The OVQ2D performance of these memory configurations built by the Object Memory Population (OMP) algorithm and processed by the Query Retrieval and Localization (QRL) algorithm are reported in Table 6. As can be noticed, the proposed memory representation (MR1*) offers the best balance between QRL accuracy and retrieval time.

Classifier implementation details The classification model was implemented using a deep neural network consisting of a ResNet18 [23] feature extractor and a linear layer, which served as the representation filter. This model was fine-tuned on a dataset of 17,071 visual representations extracted from 250 uniformly sampled clips from the training set of the Ego4D Episodic Memory VQ2D-EgoTracks benchmark [22, 46]. Specifically, the training set included 8,534 visual representations labeled as 0 (i.e., not suitable for QRL) and 5,153 representations labeled as 1 (i.e., suitable for QRL). The validation set consisted of 2,126 visual representations labeled as 0 and 1,258 representations labeled as 1.

The training and validation representations were labeled as 0 or 1 based on whether they were incorrectly or correctly matched by the QRL algorithm. For each frame in the 250 selected clips, the representation ϕ_i of the annotated visual query $\mathbf{Q}_i$ was extracted, and its feature representation $\Phi(\phi_i)$ was computed using Siam-RCNN [48] as $\Phi(\cdot)$. This representation was then compared to $\Phi(\phi_{i,t})$, the feature representation of the patches withing the ground-truth bounding boxes of the tracklet associated with the query $\mathbf{Q}_i$, using a classification head [48]. This comparison produced a similarity score s_t ranging from 0 to 1. Patches were assigned negative labels if $s_t \leq 0.3$ and positive labels if $s_t \geq 0.7$. After labeling, the data was randomly split into training and validation sets, with 80% allocated to the training set and 20% to the validation set.

OMP Configuration OD	OT	Memory Representation	%tAP$_{25}$ ↑	%stAP$_{25}$ ↑	%Succ ↑	Time ↓
OD$_{oracle}$	OT$_{oracle}$	MR1*	73.2	68.1	**81.92**	0.51 s
		MR2	**73.3**	**68.3**	**81.92**	4.27 s
		MR3	63.7	55.1	74.33	**0.05 s**
		MR4	70.4	66.2	80.35	0.32 s
OD$_{yolo}$	OT$_{oracle}$	MR1*	**21.0**	19.4	**40.55**	0.36 s
		MR2	**21.0**	**19.5**	**40.55**	2.71 s
		MR3	20.3	17.0	38.02	**0.05 s**
		MR4	20.7	17.2	39.05	0.21 s
OD$_{oracle}$	OT$_{egostark}$	MR1*	**0.18**	**0.05**	**31.91**	0.49 s
		MR2	**0.18**	**0.05**	**31.91**	4.98 s
		MR3	0.16	0.03	26.56	**0.05 s**
		MR4	0.16	0.04	29.02	0.29 s
OD$_{yolo}$	OT$_{egostark}$	MR1*	**0.03**	**0.02**	**4.02**	3.08 s
		MR2	**0.03**	**0.02**	**4.02**	14.90 s
		MR3	0.02	0.03	1.78	**0.05 s**
		MR4	0.02	0.05	2.90	0.69 s

Table 6. **Effect of different filtering strategies.** This table illustrates the impact of using different filtering approaches for $\mathcal{M}_{ego}$ population on QRL performance and retrieval time. MR1* refers to the representation described in the main paper. As can be noticed, MR1* constitutes the best approach at the intersection of QRL accuracy and retrieval efficiency. (Best results are highlighted in **bold**).

Tracker	Detector	HOTA ↑	OWTA ↑	AssA ↑	DetA ↑
MASA	GroundingDino	0.036	**0.211**	0.105	0.013
MASA	YOLOv10*	**0.066**	0.080	0.116	**0.038**
MASA	Ground-truth	0.501	0.501	0.252	1.000
EgoSTARK	Hand-Object Detector	0.034	0.121	0.102	0.012
EgoSTARK	GroundingDino	0.057	0.121	0.115	**0.029**
EgoSTARK	YOLOv10*	0.061	0.188	**0.151**	0.025
EgoSTARK	Ground-truth	0.217	0.390	0.347	0.137
BoT-SORT	YOLOWorld	0.014	0.073	0.042	0.005
Ground-truth	YOLOv10*	0.848	0.848	0.943	0.762
Ground-truth	GroundingDino	0.875	0.875	0.943	0.812

Table 7. **Evaluation of different tracking algorithms for the implementation of the Object Tracking (OT) module.** We evaluated the multi-object tracking (MOT) performance of our implmentations on EgoTracks's validation set. Models marked with "*" are finetuned on EgoTracks [46]. Ground-truth refers to the EgoTracks annotations used to simulate optimal object detection or tracking, and the respective results are reported in gray. We found the AssA results to be best indicators for QRL accuracy. (Best results, across real-world tracker-detector combinations, are highlighted in **bold**.)

The visual representation classifier was fine-tuned for 50 epochs using Stochastic Gradient Descent (SGD) with a learning rate of 0.01. At convergence, it achieved a test accuracy of 97%. The input patches were resized to 224×224 pixels before being fed into the model.

A.2. Object Memory Population (OMP)

A.2.1. Object Tracking (OT) module.

In this section, we provide details regarding the implementation of OT module. In Table 7, we report the performance of the different implementations as measured by standard multiple object tracking (MOT) metrics [32].

OT$_{egostark}$. OT$_{egostark}$ implements a multiple-object tracking approach using multiple instances of single-object trackers. This approach is motivated by the lack of effective multiple-object tracking solutions for egocentric vision, whereas single-object tracking has been extensively studied [11, 12, 46]. For single-object tracking, we use the STARK framework [52], fine-tuned on the EgoTracks dataset as described in [46], resulting in the EgoSTARK tracker. To track multiple objects, we instantiate multiple EgoSTARK trackers simultaneously. A new tracker instance is initialized in frame $\mathbf{F}_t$ whenever the OD module identifies a valid object, which is then stored in memory. Each bounding box predicted by the OD module serves as a template to initialize a corresponding tracker. Once initialized, a tracker runs across subsequent frames to predict the bounding boxes of the object it was assigned to, thereby updating the spatio-temporal representation of the respective object. Due to the challenges of object tracking in egocentric vision [11, 12, 46], we found different tracker instances to often end in tracking the same object. In order to prevent duplicate tracker instances, we calculate the IoU between all tracked objects at frame t. If two tracking predictions overlap with an IoU greater than 0.5, the trackers are considered to be duplicate and only the tracker instance and their corresponding prediction with the lower object instance ID (i.e. the older object) is retained.

Furthermore, to prevent incorrect tracking predictions from causing a tracker to deviate from a searching area that contains the target object and wander across the frame, the box used to compute the search region for the subsequent frame is not updated if the confidence score of the EgoSTARK instance falls below 0.4. This strategy is based on the observation that, in egocentric vision, objects often leave the frame during rapid head movements but typically reappear in a similar location when the head returns to its original position. Before writing the OT module predictions to $\mathcal{M}_{ego}$, the bounding boxes $\mathbf{b}_{i,t}$ with confidence score $s_{i,t} > \lambda_{ot} = 0.5$ are retained.

OT$_{masa}$. As an alternative approach, we implemented the state-of-the-art association method MASA [27] as a multi-object tracker, utilizing object detections provided by the OD module. We set a score threshold of 0.1 and limited object comparisons to the last ten frames using the MASA adapter. To evaluate the impact of considering more frames, we tested with 150 previous frames, but this led to a drop in the OWTA metric from 0.2111 to 0.1496. This result highlights the challenges the MASA adapter faces with long-term tracking.

OT$_{yoloworld}$**.** Finally, we investigated the tracking algorithm BoT-SORT [1] included in the YOLOWorld [8] implementation.

OT$_{oracle}$ We implement the oracular object tracker as follows: whenever a detected object intersects a GT bounding box from EgoTracks, we put the rest of the track (from that box onward) in memory. EgoTracks extends the Episodic Memory VQ2D benchmark introducing an object track for each query, hence GT tracks from EgoTracks are valid GT VQ2D tracks. When we combine OT$_{oracle}$ with OD$_{oracle}$, we simply insert in memory all object tracks from EgoTracks.

A.2.2. Object Discovery (OD) module

Clustering-based object taxonomy on EgoTracks. As detecting objects from egocentric videos is not straightforward [57], we tested different object detectors including the off-the-shelf detectors GroundingDino [30] and the hand-object detector of [43] (here referred to as Hand-Object Detector) which do not require training on the considered settings, as well as Faster R-CNN R101-FPN [40] and YOLOv10 [49] instances trained on different data sources, including EgoObjects [57] and curated versions of EgoTracks [46]. Targeting visual object tracking, EgoTracks contains spatially sparse bounding box annotations across contiguous frames (i.e., only few object instances are annotated in each frame). Each object instance is associated to a textual description rather than an object class (e.g., "a blue bottle"). To enable the training of object detectors, we first clustered object descriptions with K-Means to obtain an initial set of $1,169$ classes, followed by manual refinement in order to adjust them, leading to a diverse, granular taxonomy comprising 295 classes.

Data cleaning for EgoTracks. Since EgoTracks contains annotations in contiguous frames, we uniformly sampled one-third of all frames. EgoTracks contains annotations for very small objects to assess the ability of visual object trackers to continuously follow objects subject to scale changes. Since these objects would be unfeasible to detect from single frames and may be source of ambiguity due to occlusion or motion blur, we discarded bounding box annotations with areas falling in the first quartile of all bounding box areas in the training set. Figure 7 shows a few examples of annotation bounding boxes that fall in the first quartile and therefore discarded from the dataset. This leads to a reduced training set of $474,761$ annotated images, while we consider a validation set of $2,466$ images.

Faster R-CNN - based detector. A first version of the object discovery module is obtained by training a Faster R-CNN instance on the above described dataset with object class taxonomies obtained through clustering of object descriptions. The performance of this module is limited as shown later in the quantitative evaluation, so this implementation of the OD module is not considered in the main paper.

Figure 7. **Visualization of discarded bounding-boxes.** Small-area bounding boxes similar to those shown in this figure have been discarded for the training and evaluation of the module. This is because we found that they lacked sufficient visual information, leading to an increased number of false positive detections. These detections provided no useful data for tracker initialization within the OT module or for effective QRL.

YOLO - based detector. A second version of the object discovery module was based on the YOLOv10 [49] object detector, given its capacity for real-time inference. We trained the YOLOv10 detector on the same set of refined EgoTracks annotations. Given the sparsity of EgoTracks annotations, both the Faster R-CNN and YOLO detectors tend to recognize a few object instances per image, which may limit the ability of the object discovery module to identify all possible objects of interest. To address this issue, we trained the YOLOv10 object detector on the EgoObjects [57] dataset, which contains dense object annotations for egocentric frames with a fixed taxonomy of 638 classes. To make the EgoObjects and EgoTracks sets compatible, we mapped each EgoTracks object description to its corresponding class from the EgoObjects taxonomy. This is done by first describing the object description and class name with a *Sentence Transformer*, and then selecting the class maximizing the cosine similarity between the embeddings. To reduce noise, we discarded all annotations with a similarity lower than 0.7.

Examples of class mapping. Table 8 shows some examples of object descriptions retrieved from the EgoTracks annotations and mapped to EgoObjects objects. Note that some objects such as "air conditioner", "headphones" or "yoga mat" have a reasonable match, while "washing machine" contains "flat washer", which has a different meaning despite a close embedding. We did not manually adjust

EgoObjects Classes	accordion	air conditioner	headphones	washing machine	...	yoga mat
EgoTracks Mapped Classes	-	air conditioner, air conditioner control, air conditioner., air conditioning machine	head phones, headphone, headphones, headset, earphone, earphones	flat washer, washing machine	...	gym mat, mat, workout mat, yoga mat, exercise mat, fitness mat

Table 8. **Examples of class mappings from EgoObject classes to EgoTracks object descriptions.** The first row reports EgoObject classes, while the second row contains descriptions of EgoTracks matched with embedding similarity.

these inconsistencies to keep the process fully automated. The table contains one class with no matches (accordion), due to the absence of EgoTracks objects with similar descriptions. Out of the total number of class descriptions (3491), we mapped 1,241 captions, while the remaining 2,250 classes were excluded from the dataset. This resulted in a high-quality subset of the data suitable for training and evaluation.

Quantitative comparison. Despite the domain difference between EgoObjects and EgoTracks, we noticed improved qualitative and quantitative results. To further adapt the model to the EgoTracks domain, we fine-tuned the YOLOv10 object detector, pre-trained on EgoObjects, on the EgoTracks dataset with labels mapped to the EgoObjects taxonomy. Table 9 compares the performance of the considered approaches with three off-the-shelf object detectors: GroundingDino [30], the Hand-Object-Detector of [43], and YoloWorld [8]. We do not train these detectors on the EgoTracks data and use the official checkpoints and implementations provided by the authors. Since Visual Query Localization does not require to correctly identify object classes, all methods are evaluated with AP and Recall, merging all ground truth and predicted classes in a single "object" class. It is worth noting that AP and Recall measure different aspects of the detector. Indeed, AP measures the average ratio between correct predictions and all predictions made by the detector, whereas Recall measures the fraction of objects which have been correctly found by the detector, regardless of the total number of objects. In our settings, it is desirable to have a large recall (future queries have more chance to end up in the memory), but, since it is not possible to track all objects and a selection must be made based on the confidence score, in practice, a very low value of AP is not desirable. The harmonic mean between AP and Recall is hence adopted as a way to identify the most balanced detector. Results in Table 9, obtained with a confidence score threshold $\lambda_{od} = 0.01$, show that Faster R-CNN trained on labels clustered from EgoTracks achieves the second-highest recall (0.5224), but also yields a very low AP value (0.0219). On the contrary, the YOLOv10 instance achieves the highest AP (0.1983) but also a low recall (0.1634). We attribute these skewed results to the sparse nature of annotations in EgoTracks (only a few object instances annotated per frame) and the different ways the two methods handle negative examples. The harmonic mean results of the first one are rather low (0.0420), while the second result is higher but not optimal at 0.1791, highlighting the unbalanced nature of the two approaches. Training YOLOv10 on EgoObjects, which contains much denser annotations increases Recall to 0.3780 at the expense of AP (0.0247), which leads to a low harmonic mean of 0.0463. Mapping EgoTracks annotations to the EgoObjects taxonomy and training YOLOv10 using both datasets provides the most balanced detector, with a harmonic mean of 0.1882, an AP of 0.1358 and a Recall of 0.3067.

YOLOWorld achieves the lowest precision value among all the methods (0.0207), but the highest recall, equal to 0.5969. Due to the low precision value, the harmonic mean has a low score equal to 0.0401. GroundingDino achieves a low AP and a reasonable recall value (0.0450 and 0.2572), which is likely due to the large domain shift between data used for training GroundingDino and the challenging egocentric observations of EgoTracks. Using a Hand-Object-Detector as an object detector leads to lower results, which is because the Hand-Object-Detector only focuses on interacted objects, hence neglecting many non-interacted objects which may nevertheless be queried in the future. Lastly, RT-DETR (used in VideoAgent [15]) achieves results comparable to GroundingDino, due to similar reasons.

Qualitative Examples. Figure 8 reports qualitative examples, reporting the predictions of the discussed models on a frame from the validation set of EgoTracks. Results are consistent with the evaluation metrics in Table 9, Faster R-CNN (A) is the model with the second-highest recall, indeed it is the model with the highest number of bounding boxes containing both correctly localized objects and errors. YOLOv10 trained on EgoTracks (B) has fewer predictions than Faster R-CNN, but they are more accurate. Training YOLOv10 on EgoTracks (C) shows similar results to Faster R-CNN, with the second-densest predictions, translated into the second-highest recall model. YOLOv10 trained on EgoObjects fine-tuned in EgoTracks (D) has the most balanced results; predictions are well localized, reflecting the best-performing model in the HM metric. YOLOWorld (E)

Detector	Training Dataset	AP ↑	Rec ↑	HM ↑
Faster R-CNN	EgoTracks*	0.0219	0.5224	0.0420
YOLOv10	EgoTracks*	**0.1983**	0.1634	0.1791
YOLOv10	EgoObjects	0.0247	0.3780	0.0463
YOLOv10	EgoObjects + EgoTracks**	0.1358	0.3067	**0.1882**
YOLOWorld	-	0.0207	**0.5969**	0.0401
GroundingDino	-	0.0450	0.2572	0.0765
Hand-Object Detector	-	0.0247	0.0381	0.0299
RT-DETR	-	0.0418	0.3374	0.0743

Table 9. **Comparison of the different object detectors.** * denotes that the object taxonomy has been obtained through clustering. ** denotes that classes have been obtained by mapping EgoTracks descriptions to the EgoObject taxonomy. HM: Harmonic Mean between AP and Recall. (Best results are highlighted in **bold**.)

achieves results comparable to Faster R-CNN (A), demonstrating the highest recall with dense object predictions and minimal errors, which can be further reduced by increasing the threshold. iGroundingDino (F) contains several correctly regressed objects, but as shown in the frame, no ground truth annotations were found. Hand-Object Detector (G) has the worst performance, containing only two predictions, one of which is related to the interacted object. The discovery mechanism of this detector leads to the lowest values for both precision and recall. The predictions of VideoAgent (H) are partially correct, despite the presence of a few wrong bounding boxes. Finally the predictions marked with I are the ground truth ones.

OD$_{yolo}$ and OD$_{gdino}$. Given these results, in the following experiments, we consider the YOLOv10 detector trained on EgoObjects and EgoTracks as a representative of trained methods and both GroundingDino and Hand-Object-Detector as representative of off-the-shelf methods. These are referred to as OD$_{yolo}$ and OD$_{gdino}$ respectively in the main paper.

OD$_{oracle}$. The oracular object detector is obtained using the ground truth object annotations in EgoTracks as valid detections. As previously mentioned, EgoTracks is aligned to the Episodic Memory benchmark of Ego4D, so EgoTracks detections include valid queries.

A.3. Query Retrieval and Localization (QRL)

We implemented two distinct approaches for localizing a visual query $\mathbf{Q}$ within a memory $\mathcal{M}_{ego}$: an implementation based on SiamRCNN with a classification head [48](QRL$_{siamrcnn}$); and an approach based on DINOv2 with cosine similarity [21, 34] (QRL$_{dino}$).

In QRL$_{siamrcnn}$, a Feature Pyramid Network (FPN) [28] serves as the backbone $\Phi(\cdot)$ for features extraction, generating features for the visual representation of the object $\Phi(\phi_{i,t}), \phi_{i,t} \neq \emptyset$ stored in $\mathcal{M}_{ego}$ as well as the features

Figure 8. **Visualization of predictions from the object detection models.** Here we qualitatively show the performance of the different object detectors over the same sampled frame. A: Faster R-CNN on EgoTracks, B: YOLOv10 on EgoTracks, C: YOLOv10 on EgoObjects, D: YOLOv10 on EgoObjects, fine-tuned in EgoTracks, E: YOLOWorld, F: GroundingDino, G: is the Hand-Object Detector H: RT-DETR, I: Ground-truth annotations.

for the visual query $\Phi(\mathbf{Q})$. To determine whether $\mathbf{Q}$ exists within $\mathcal{M}_{ego}$, each $\Phi(\phi_{i,t})$ is compared with $\Phi(\mathbf{Q})$ using a siamese network head, $\Psi(\Phi(\phi_{i,t}), \Phi(\mathbf{Q}))$, which implements a bilinear operation [48] that predicts instance similarity score belonging to the interval [0, 1].

In QRL$_{dino}$, DINOv2 [34] is employed as the backbone $\Phi(\cdot)$ for extracting feature representations $\Phi(\phi_{i,t})$ and $\Phi(\mathbf{Q})$. In this approach, the similarity operation $\Psi(\Phi(\phi_{i,t}), \Phi(\mathbf{Q}))$ is implemented using cosine similarity [21], which provides scores belonging to the interval [0, 1].

In both cases, for each object i, the similarity scores are averaged into a single score r_i. If the maximum score r_i exceeds the threshold λ_{ref} = 0.5, the object $\mathbf{O}_i$ is considered a match for query $\mathbf{Q}$. Once a match is identified, the most recent sequence of contiguous bounding boxes from $\mathbf{O}_i$'s spatio-temporal history is retrieved and returned as the response track $\mathbf{r}$.

A.4. ESOM Memory Overhead

The GPU memory overhead is 423MB for OD_{yolo} and 2,350MB for $\text{OT}_{egostark}$ when tracking 10 objects concurrently, 2,320MB for OT_{masa}, and 2,474MB for $\text{QRL}_{siamrcnn}$.

B. Online Object Memory Baselines

Here we provide details of the implementations of the baselines used for comparison.

B.1. AMEGO

The AMEGO framework [21], was employed to construct an episodic memory model, serving as a performance benchmark for our proposed approach. We utilized *ce683bd* version of the AMEGO codebase, available at , with minor modifications to accommodate the EGO4D dataset structure. Our implementation focused specifically on the Hand-Object Interaction (HOI) module of AMEGO. The Location Segmentation module was excluded from our analysis as it fell outside the scope of our research objectives. To ensure a fair and consistent comparison, we maintained the original hyperparameters and thresholds to align with the baseline model published on the official GitHub repository. Since their experiment has been made using a different dataset, our modifications were limited to data input/output functions to handle the EGO4D data format, without altering the core algorithmic components.

B.2. VideoAgent

The VideoAgent framework [15], has been utilized as a comparative baseline for our episodic memory model. We employed the *21e4eb7* version of the VideoAgent repository. Our implementation focused on the Object Memory module, and we opted to omit the captioning module since it was not pertinent to our research scope. To maintain experimental integrity, we preserved the original configuration parameters.

B.3. Memory Representation

As with the other OMP configurations discussed in this paper, the object tracklets produced by VideoAgent and Amego OMP are stored in memory using the MR1* representation, as it provides the best balance between accuracy and memory efficiency (see Table 6). Specifically, for each $\mathbf{O}_i$ found by either AMEGO or VideoAgent object detection and tracking algorithms, $\mathcal{M}_{ego}$ retains the RGB patch ($\phi_{i,t}$) and the $\mathbf{b}_{i,t}$ at the time of object discovery, along with two RGB patches from the most recent response track. These patches correspond to the first and last valid patches identified by the classifier within the most recent sequence of bounding boxes. For all other temporal locations not covered above, $\mathcal{M}_{ego}$ stores only the spatio-temporal information provided by $\mathbf{b}_{i,t}$.

https://github.com/gabrielegoletto/AMEGO.git
https://github.com/YueFan1014/VideoAgent.git

OMP Configuration OD	OT	%tAP$_{25}$ ↑	%stAP$_{25}$ ↑	%Succ ↑	Size ↓
OD_{oracle}	$\text{OT}_{egostark}$	**18.3**	**5.1**	**31.91**	**591.4 MB**
OD_{yolo}	$\text{OT}_{egostark}$	0.03	0.2	4.02	1.7 GB MB
$\text{OD}_{oracle+yolo}$	$\text{OT}_{egostark}$	10.6	2.3	23.67	2.4 GB

Table 10. **More objects in the memory can confound QRL.** In this table, by combining the capabilities of the OD_{oracle} and OD_{yolo}, we tested wether the QRL method is robust to a memory $\mathcal{M}_{ego}$ which contains the true visual queries as well as other object found while processing $\mathcal{V}_{ego}$. (Best results are in **bold**).

B.4. Query Retrieval and Localization (QRL)

To localize the visual queries $\mathbf{Q}$ within the memory $\mathcal{M}_{ego}$ built over AMEGO and VideoAgent, we used the implementation based on SiamRCNN with a classification head ($\text{QRL}_{siamrcnn}$). Once the similarity scores are computed for the different patches, they are averaged into a single score r_i. If the maximum score exceeds the λ_{ret} = 0.5, the object $\mathbf{O}_i$ is considered a match for query $\mathbf{Q}$.

C. Evaluation Dataset

Due to the high storage requirements for the validation split of the Ego4D VQ2D benchmark [22] and the significant time needed for experiments on the full dataset, we opted to use a subset of the validation clips for our evaluations. Specifically, we randomly selected 115 video clips uniformly, containing 450 visual queries along with their corresponding ground-truth response tracks and tracklets. This subset consisted of a total of 208,365 frames. To maintain consistency with the original frame rate of the EGO4D dataset [22], we ran the OMP algorithm at 5 frames per second.

D. Additional Results

Combination of OD modules. The performance of ESOM has been evaluated under various configurations of the OD and OT modules (Table 5). However, OMP configurations using OD_{oracle} construct a memory that includes only objects that will be queried. To incorporate addi-

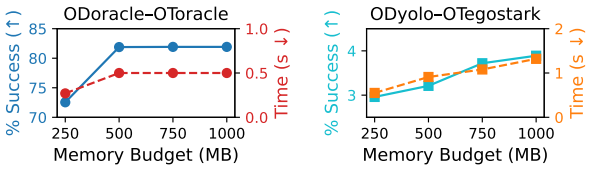

Figure 9. **Impact of memory-budget on ESOM performances.** Success rate and query time for two ESOM variants under varying memory budgets (250–1000 MB). Results show that performance remains stable as older or low-confidence objects are pruned to meet memory constraints, indicating ESOM's robustness under limited memory conditions.

tional legitimate objects into $\mathcal{M}_{ego}$, we combined the oracle detections, together with the OD_{yolo}detections (see $\text{OD}_{oracle+yolo}$ in Table 10). Adding more objects to memory reduces ESOM's performance compared to the OMP configuration with OD_{oracle}. This suggests that an increased number of objects in memory hinders the QRL algorithm's ability to accurately discriminate the visual query, resulting in confusion. However, its performance remains higher than the OMP configuration using only OD_{yolo}, further highlighting the challenges detectors face in identifying objects that will appear as queries in the future.

Random performance The performance of ESOM has been evaluated by implementing a QRL method that randomly (w. uniform dist.) selects a response track from $\mathcal{M}_{ego}$. This achieves a Succ. score of 27.29 (OD_{oracle}+OT_{oracle}) and of 0.44 (OD_{yolo}+$\text{OT}_{egostark}$), while ESOM achieves 81.92 and 4.02 respectively. ESOM performance, even in suboptimal OMP configuration (OD_{yolo}+$\text{OT}_{egostark}$) are still better than random selection. This indicates that the method effectively leverages available information to enhance performance.

Constant memory budget We explored a "constant-budget" version of ESOM, in which older and low-confidence objects are pruned as soon as a predefined memory limit is reached. Results (Figure 9) demonstrate that ESOM isn't particularly affected by memory constraints.

Qualitative examples. Figure 10 shows additional qualitative examples of the behavior of QRL in retrieving and localizing different visual queries within two memory representations build by two OMP configurations.

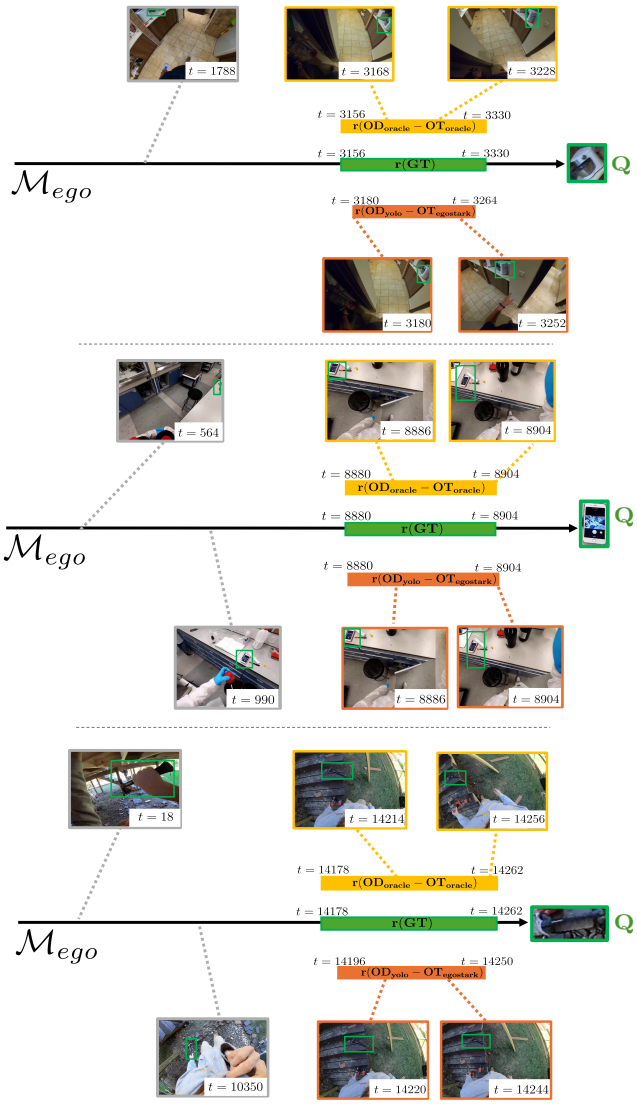

Figure 10. **Additional qualitative examples.** Successful retrieval of thre different queries Q from $\mathcal{M}_{ego}$ using the oracle OMP algorithm (OD_{oracle} - OT_{oracle}) and the real-world OMP (OD_{yolo} - $\text{OT}_{egostark}$). Sample frames from each track are shown with corresponding bounding boxes whose patches $\phi_{i,t}$ were matched during retrieval. The frames in gray correspond to the moment of the object's discovery. The green bar visualizes the temporal extent of the ground-truth track, with timestamps indicating its duration.